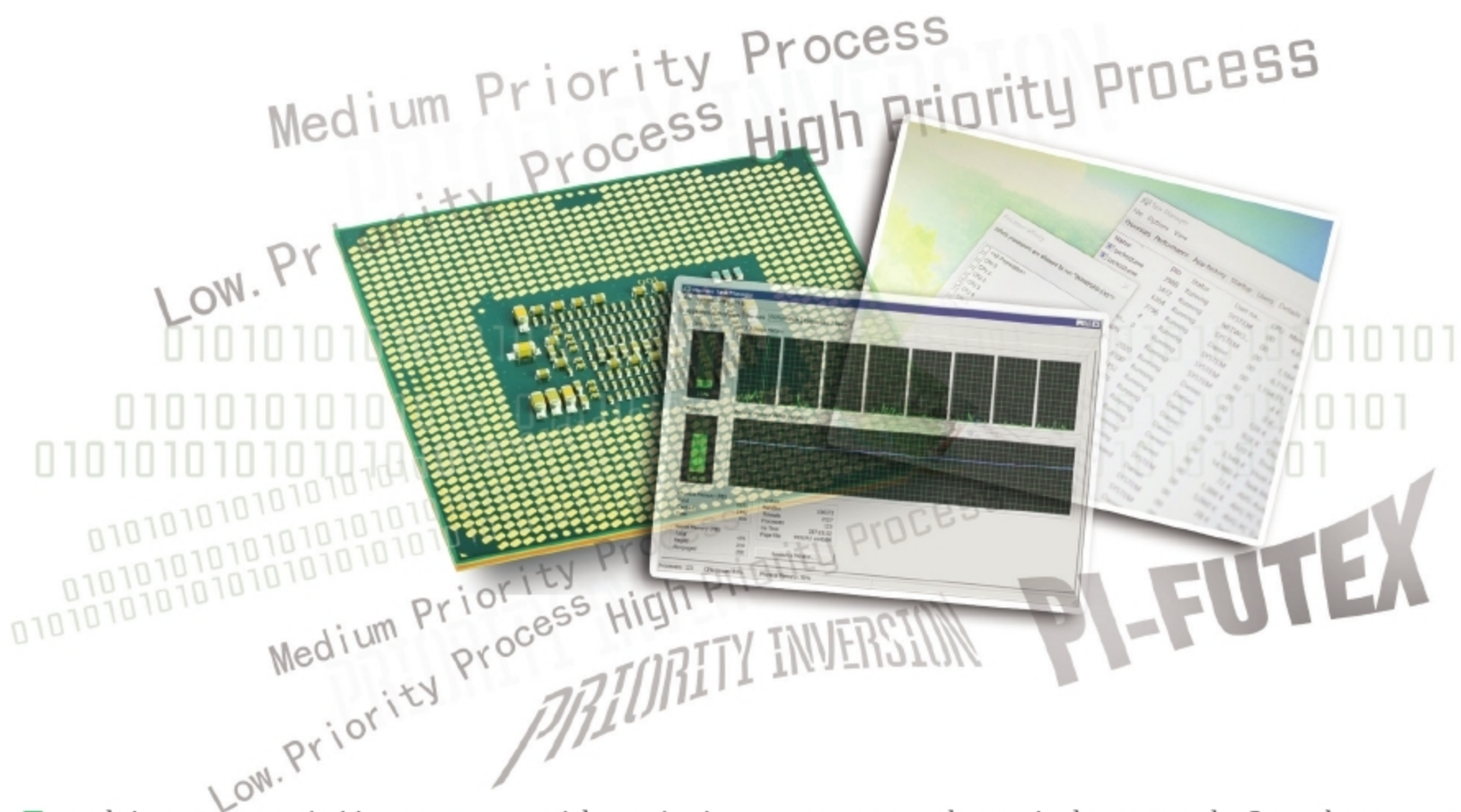


Get Rid of Priority Inversion with PI-Futex

This article helps the reader understand the priority inversion problem of process scheduling at the user space level and provides solutions for it. Also discussed are priority inversion with a semaphore, avoiding priority inversion by using different CPU affinities for processes, and priority inheritance with PI-Futex.



In real-time systems, priorities are very essential to maintain predictability. The scheduler of these systems ensures that the higher priority task should be executed before a low priority task. For this purpose, high priority tasks may preempt low priority tasks. But as tasks share resources, those independent of a particular resource but with a lower priority can prevent the highest priority ready task from running when it should. This is 'priority inversion'. Due to this, a critical deadline could be missed and systems might fail.

This problem was experienced by the Mars Pathfinder spacecraft. It landed on Mars and began to transmit data back to the earth. Days later, the flow of information and images was interrupted by a series of total systems resets. The source of the problem was priority inversion, which subsequently caused a critical task missing a deadline. This was identified by a watchdog timer and finally, the only solution was to reset the spacecraft — a short C program was uploaded to the spacecraft, which when interpreted, changed the values of the mutex flag for priority inheritance from false to true. No more systems resets occurred!

In order to achieve locking, a semaphore can be used — it provides synchronisation by restricting access of shared

resources to the required process only. Semaphores are non-negative integer values that support two atomic operations, *semaphore-P()* and *semaphore-V()*. P allows waiting for a semaphore to be positive and then decrements it by one, while V allows incrementing a semaphore by one, which implies it wakes up a waiting P. But a semaphore causes the priority inversion problem.

To resolve the priority inversion problem Futex is used. It provides priority inheritance, which is explained later.

An experimental setup

We used a 64-bit, four-core Ubuntu 16.04 LTS system with i686 architecture to conduct the experimental work.

All the code for this setup is available at <https://github.com/DhanashreeMohite/ProjectElectiveWork>.

Priority inversion with a semaphore

sem.c creates a binary semaphore. Here, the *ftok* function returns the key depending on the current path files and ID 'a'. The *semget* creates a semaphore identifier associated with a key. The *semctl* function sets the value of *semval* to *arg.val* for the 0th semaphore of the set.